

ORDER BY y LIMIT: Cómo ordenar y paginar resultados en SQL

Hasta ahora las consultas traen datos, pero sin un orden definido. El motor los devuelve en el orden en que los encontró, que puede ser cualquiera. Para responder preguntas de negocio reales, el orden importa: no es lo mismo una lista de productos que los diez productos más caros, ni un listado de pedidos que los tres pedidos más recientes.

ORDER BY y **LIMIT** son los dos comandos que convierten una consulta genérica en una respuesta concreta a una pregunta de negocio.

ORDER BY: definir el orden de los resultados

ORDER BY va al final de la consulta y recibe el nombre del campo por el que quieres ordenar.

```
SQL
-- De menor a mayor precio (ascendente):

SELECT nombre, precio
FROM productos
ORDER BY precio ASC;

-- De mayor a menor precio (descendente):

SELECT nombre, precio
FROM productos
ORDER BY precio DESC;
```

ASC es ascendente: de menor a mayor, de A a Z, de la fecha más antigua a la más reciente. Es el orden por defecto, así que si no escribes nada, el motor asume **ASC**.

DESC es descendente: de mayor a menor, de Z a A, de la fecha más reciente a la más antigua.

ORDER BY con múltiples columnas

Puedes ordenar por más de un campo a la vez. El motor aplica el primer criterio y, cuando hay empate, aplica el segundo.

```
SQL
SELECT nombre, categoria_id, precio

FROM productos

ORDER BY categoria_id ASC, precio DESC;
```

Esto agrupa los productos por categoría de menor a mayor ID, y dentro de cada categoría los ordena de mayor a menor precio. Es útil para cualquier análisis que necesite un orden jerárquico: primero por región, luego por ventas; primero por fecha, luego por monto.

LIMIT: traer solo los resultados que necesitas

LIMIT restringe cuántas filas devuelve la consulta. Combinado con **ORDER BY**, te permite responder preguntas de negocio tipo "los N más..." o "los N menos...".

```
SQL
-- Los cinco productos más caros:

SELECT nombre, precio

FROM productos

ORDER BY precio DESC

LIMIT 5;

-- Los tres pedidos más recientes:

SELECT pedido_id, cliente_id, fecha

FROM pedidos
```

```
ORDER BY fecha DESC  
  
LIMIT 3;
```

El orden importa: primero **ORDER BY** define el criterio, luego **LIMIT** corta el resultado. Sin **ORDER BY**, **LIMIT** trae filas en orden arbitrario y el resultado no tiene sentido de negocio.

OFFSET: paginar los resultados

OFFSET le dice al motor desde qué fila empezar. Combinado con **LIMIT**, te permite dividir un resultado grande en bloques del mismo tamaño, lo que se conoce como paginación.

```
SQL  
-- Página 1: filas 1 a 10  
  
SELECT nombre, precio  
  
FROM productos  
  
ORDER BY nombre ASC  
  
LIMIT 10 OFFSET 0;  
  
  
-- Página 2: filas 11 a 20  
  
SELECT nombre, precio  
  
FROM productos  
  
ORDER BY nombre ASC  
  
LIMIT 10 OFFSET 10;  
  
  
-- Página 3: filas 21 a 30
```

```
SELECT nombre, precio
FROM productos
ORDER BY nombre ASC
LIMIT 10 OFFSET 20;
```

OFFSET 0 empieza desde el principio. **OFFSET 10** se salta las primeras 10 filas y empieza desde la 11. **OFFSET 20** se salta las primeras 20 y empieza desde la 21.

¿Cuándo es útil esto? Cuando el resultado tiene muchas filas y necesitas exportarlo o mostrarlo en partes: páginas de un reporte, lotes de exportación a Excel o CSV, resultados de una pantalla de sistema.

Cómo se comportan los valores nulos con ORDER BY

Cuando una columna tiene filas con **NULL**, el motor los trata de una forma específica al ordenar:

- Con **ASC**, los nulos aparecen **al final**.
- Con **DESC**, los nulos aparecen **al inicio**.

```
SQL
-- Nulos al final:

SELECT nombre, precio FROM productos ORDER BY precio ASC;

-- Nulos al inicio:

SELECT nombre, precio FROM productos ORDER BY precio DESC;
```

No es un error: es el comportamiento por defecto de PostgreSQL. Si necesitas controlar dónde aparecen los nulos independientemente del orden, puedes usar **NULLS FIRST** o **NULLS LAST**:

```
SQL
SELECT nombre, precio
FROM productos
ORDER BY precio ASC NULLS LAST;
```

Las preguntas de negocio que ya puedes responder

Con `ORDER BY`, `LIMIT` y `OFFSET` combinados, puedes construir consultas que respondan directamente preguntas como estas:

Pregunta de negocio	Cómo construirla
¿Cuáles son los 10 productos más caros?	<code>ORDER BY precio DESC LIMIT 10</code>
¿Cuáles son los 5 clientes más recientes?	<code>ORDER BY fecha_registro DESC LIMIT 5</code>
¿Cuáles fueron los últimos 3 pedidos?	<code>ORDER BY fecha DESC LIMIT 3</code>
¿Cómo exporto los productos de 10 en 10?	<code>LIMIT 10 OFFSET 0</code> , luego <code>OFFSET 10</code> , etc.
¿Qué productos hay en cada categoría por precio?	<code>ORDER BY categoria_id ASC, precio DESC</code>

Hasta ahora todas las consultas trabajan con una sola tabla. Pero las preguntas de negocio más interesantes cruzan información entre varias: qué compró cada cliente, qué empleado atendió cada pedido, qué productos pertenecen a qué categoría. Para eso existe `JOIN`, que es el siguiente paso.